



CHAPTER 10

Object Oriented Programming



Solving a problem by creating **object** is one of the most popular approaches in programming.

This is called **object-oriented programming**.

This concept focuses on using reusable code (**DRY Principle**).

DRY Principle
Don't Repeat
Yourself

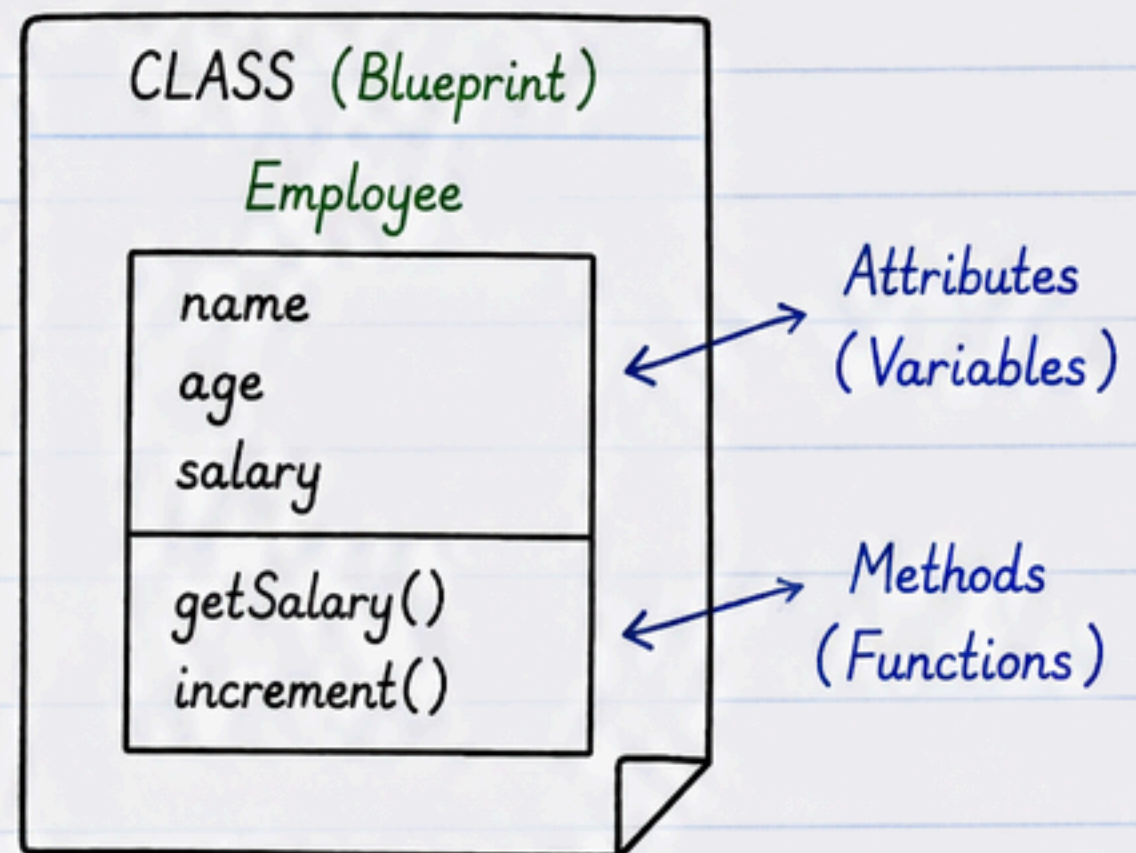


Class

A class is a **blueprint** for creating object.

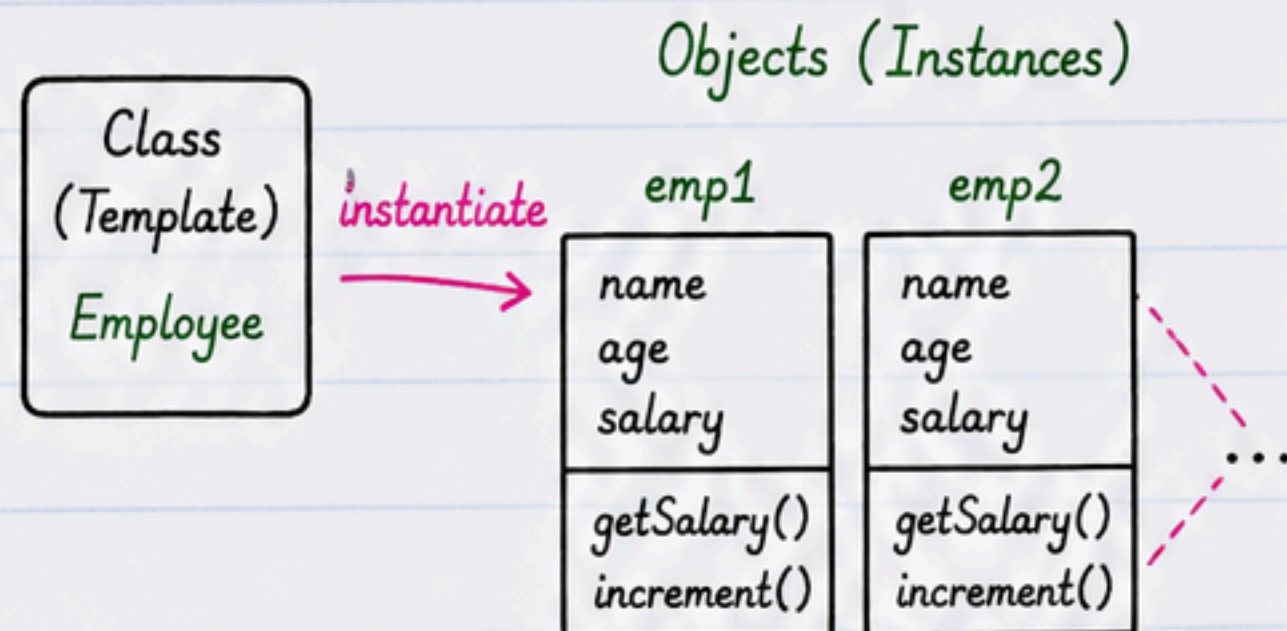
Syntax:

```
class Employee: # Class name is  
                written in pascal case  
                # Methods & Variables
```



Object

An object is an **instantiation** of a class. When class is defined, a template (info) is defined. Memory is allocated only after object instantiation.



Objects of a given class can invoke the methods available to it without revealing the implementation details to the user. - **Abstractions & Encapsulation!**



Data (attributes)
is hidden inside.
Only methods
are visible.

Modelling A Problem In OOPS

We identify the following in our problem.

- Noun → Class → Employee
- Adjective → Attributes → name, age, salary
- Verbs → Methods → getSalary(), increment()

Think like
a real world
problem!



Class Attributes

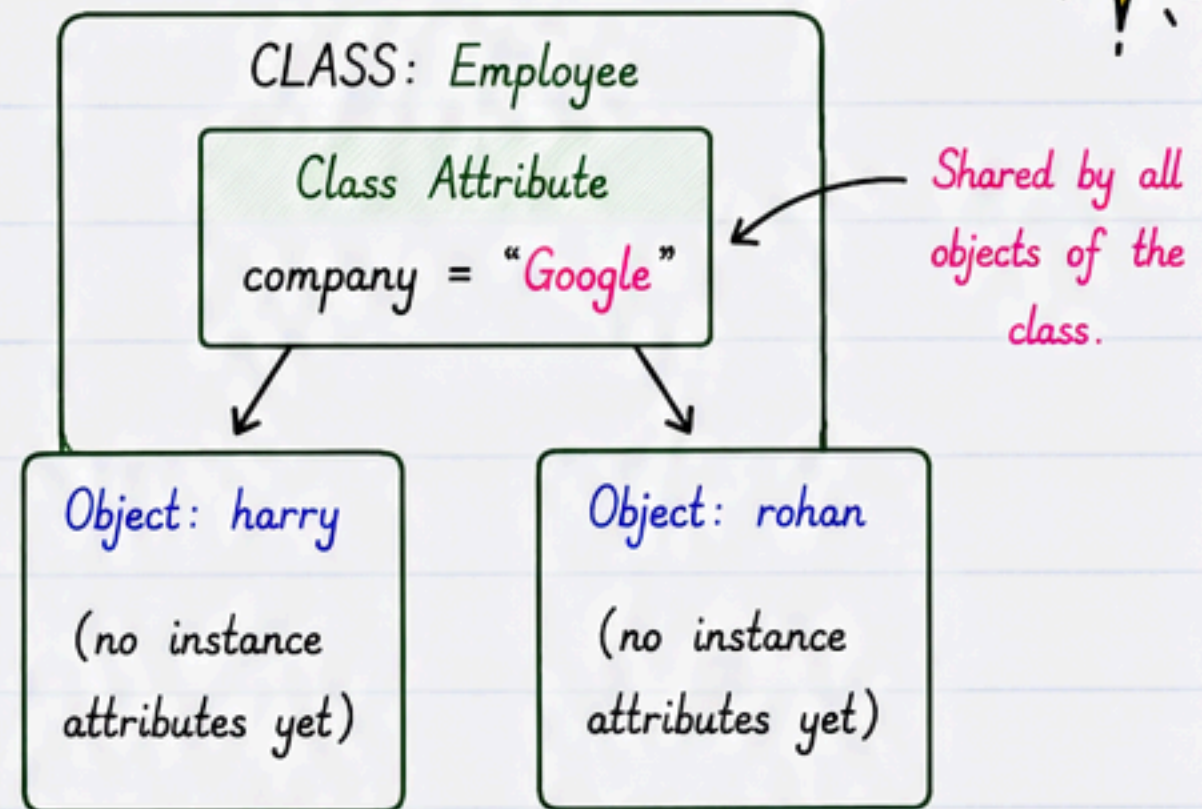
An attribute that belongs to the class rather than a particular object.

Example:

```
class Employee:
    company = "Google" # Specific to Each Class

harry = Employee() # Object Instantiation
harry.company

Employee.company = "YouTube" # Changing Class Attribute
```

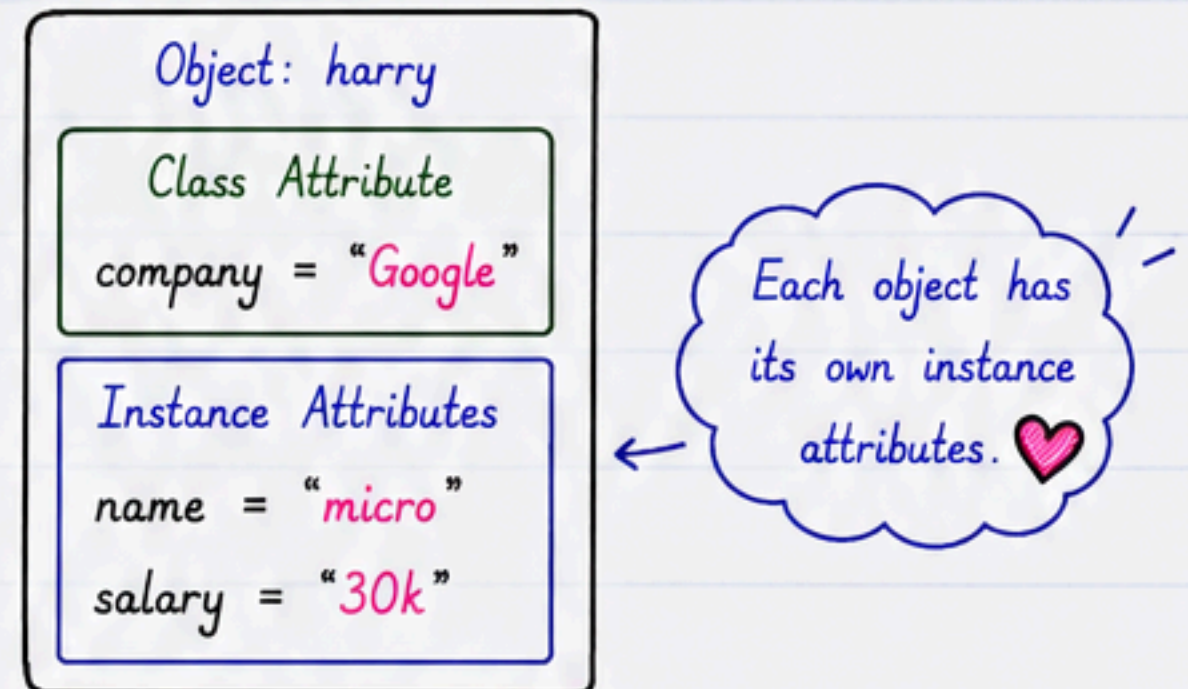


If we change the class attribute, it is reflected for all objects. ❤️

Instance Attributes

An attribute that belongs to the Instance (object). Assuming the class from the previous example:

```
harry.name = "micro"
harry.salary = "30k" # Adding instance attribute
```



★ **Note:** Instance attributes take preference over class attributes during assignment & retrieval.

When looking up for `harry.attribute` it checks for the following:

- 1) Is attribute present in object?
- 2) Is attribute present in class?



Lookup Order

1. Object (Instance)
2. Class

Self Parameter

`self` refers to the instance of the class. It is automatically passed with a function call from an object.

```
harry.getSalary() # here self is harry
# equivalent to Employee.getSalary(harry)
```

The function `getSalary()` is defined as:

```
class Employee:
    company = "Google"
    def getSalary(self):
        print("Salary is not there")
```



calls

Employee Class

`getSalary(self)`
`self → harry`

`self` acts as a reference to the current object.

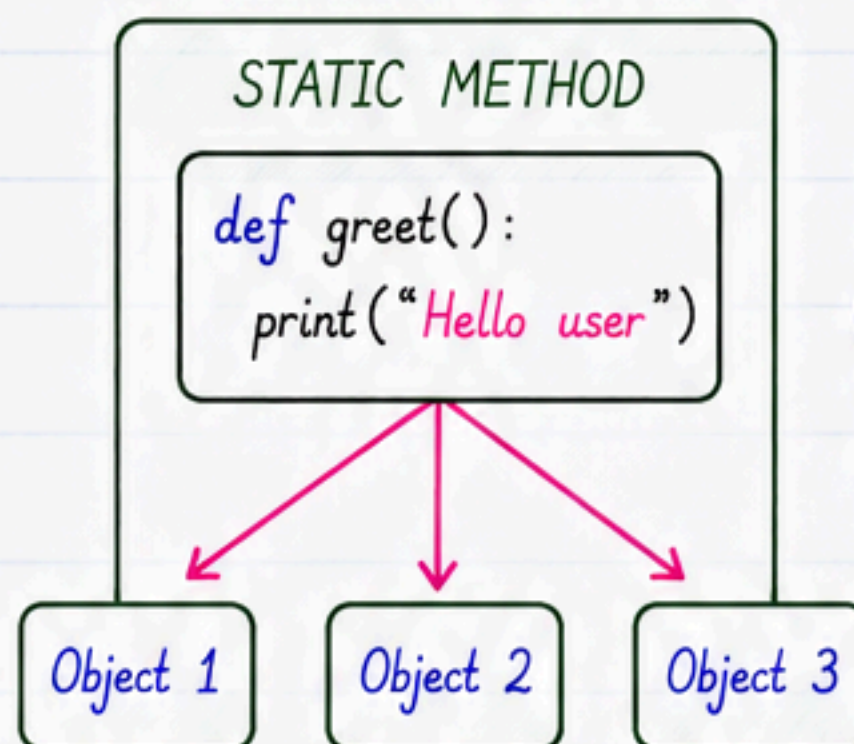


Static Method



Sometimes we need a function that does not use the **self-parameter**. We can define a **static method** like this:

```
@staticmethod # decorator to mark  
greet as a static method  
  
def greet():  
    print("Hello user")
```



Static method belongs to the class and can be called without creating object.



Call static method:

`Employee.greet()` # No object needed!



`__init__()` Constructor

- ★ `__init__()` is a special method which is first run as soon as the object is created.
- ★ `__init__()` method is also known as **constructor**.
- ★ It takes 'self' argument and can also take further arguments.

For Example:

```
class Employee:  
    def __init__(self, name):  
        self.name = name  
    def getSalary(self):  
        ...
```

`harry = Employee("Micro")`

Object created, `__init__()` is called automatically!

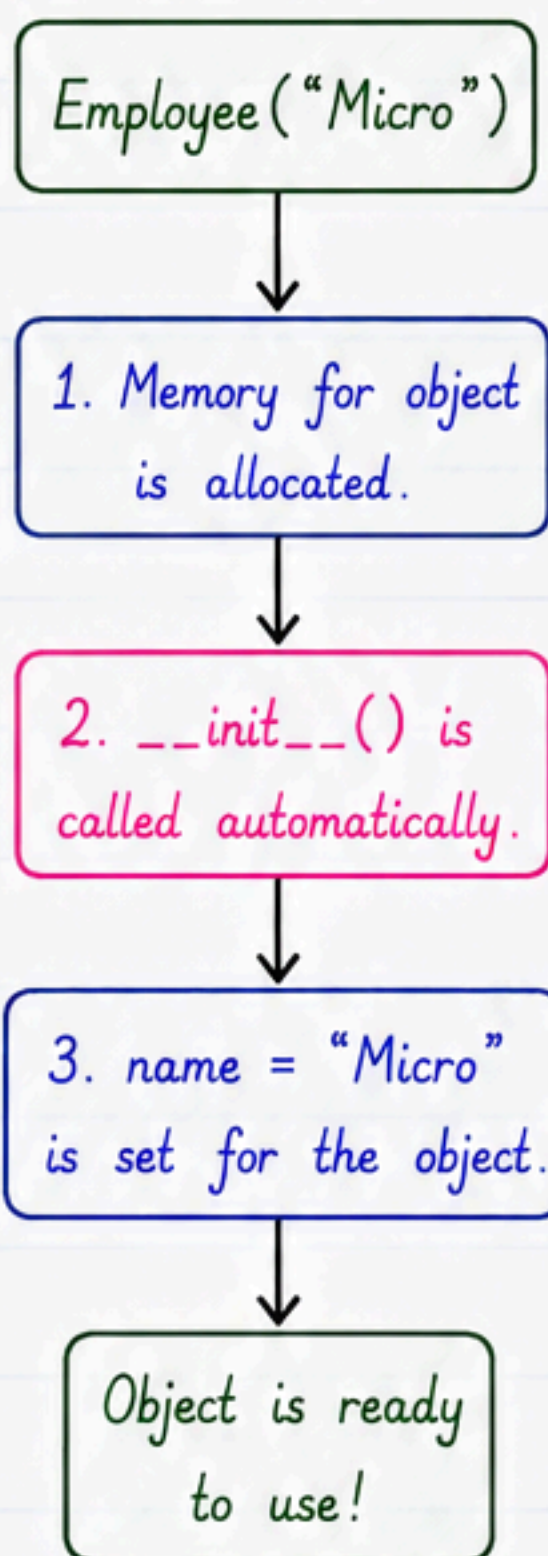


Constructor helps in initializing the object with given values. ❤️



Every class in Python has a default `__init__()` method even if we don't write it.

How it works?



Note:

`__init__()` is called only once, at the time of object creation.



Practice Set



1. Create a class "Programmer" for storing information of few programmers working at Microsoft.



Microsoft

CLASS: Programmer

Attributes
• name • language • salary
Methods
• showInfo() • getSalary()

Objects

p1 (at Microsoft)

p2 (at Microsoft)

⋮

2. Write a class "Calculator" capable of finding square, cube and square root of a number.

CLASS: Calculator

Methods
• square(n) • cube(n) • sqrt(n)

Example:

calc = Calculator()

calc.square(5)

calc.cube(5)

calc.sqrt(25)



3. Create a class with a class attribute `a`; create an object from it and set 'a' directly using `object.a = 0`. Does this change the class attribute?

CLASS: MyClass

Class Attribute
a = 10

obj = MyClass()

↓
obj.a = 0

Does this change class attribute 'a'?



4. Add a static method in problem 2, to greet the user with hello.

CLASS: Calculator

Methods
• square(n) • cube(n) • sqrt(n)
@staticmethod def greet(): print("Hello user")

Example:

Calculator.greet()

Output:

Hello user



5. Write a Class 'Train' which has methods to book a ticket, get status (no of seats) and get fare information of train running under Indian Railways.

CLASS: Train

Attributes	Methods
• trainNo • name • seats • fare	• bookTicket() • getStatus() • getFareInfo()



INDIAN RAILWAYS

6. Can you change the `self`-parameter inside a class to something else (say "harry")? Try changing `self` to `self` or `harry` and see the effects.

Example:

```
class Test:
    def hello(self):
        print("Hello")

t = Test()
t.hello()
```

Try replacing self with self or harry.

What happens?



Practice OOP daily,
Think in Objects,
Code like a Pro! ❤️

